Project Report

on

MargDarshak: Dynamic Graph Navigation Engine

A Geospatial Shortest-Path Route Navigation System with Dijkstra Algorithm & Binary Min-Heap Implementation

Data Structures and Algorithms (DSA) – Mini Project

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

Computer Science and Engineering

Submitted by:

Subham Kumar – 25BCS10000

Course: B.E. Computer Science and Engineering

Semester: 3

Under the Guidance of

Er. Kapil (E18497)

INSTITUTE – UIE

Chandigarh University

Academic Session: 2025–2026

INDEX

S. No.	Content	Page No.
	ABSTRACT	3
1	**CHAPTER 1: INTRODUCTION**	4
1.1	Background and Motivation	4
1.2	Problem Statement	4
1.3	Objectives and Scope	4
2	**CHAPTER 2: DSA BACKGROUND**	5
2.1	Graph Representation (Adjacency List vs Adjacency Matrix)	5
2.2	Priority Queue & Binary Min-Heap	5
2.3	Dijkstra’s Single-Source Shortest Path Algorithm	6
2.4	State Tracking Data Structures (Set & Map)	6
2.5	Comparison of the Data Structures	7
3	**CHAPTER 3: DESIGN AND METHODOLOGY**	8
3.1	Technologies Used	8
3.2	System Architecture and Working Flow	8
3.3	Data Structure Design and Operations	9
3.4	Algorithm Complexity Analysis	10
4	**CHAPTER 4: IMPLEMENTATION, TESTING AND RESULTS**	11
4.1	Implementation Components	11
4.2	Test Cases and Results	11
4.3	Screenshots and Observations	13
4.4	Limitations	15
5	**CHAPTER 5: CONCLUSION AND FUTURE SCOPE**	16
5.1	Conclusion	16
5.2	Future Enhancements	16

ABSTRACT

MargDarshak (मार्गदर्शक — *The Visionary Pathfinder*) is an academic geospatial route calculation engine developed for the Advanced Data Structures and Algorithms (ADSA/DSA) course. The application translates India's extensive national highway infrastructure into an active, weighted topological graph comprising 135+ major vertices (cities/junctions) and 207+ bidirectional edges, covering the Golden Quadrilateral, North-South and East-West corridors, and regional arterial roads.

The core algorithmic engine is engineered strictly from foundational computer science principles without relying on third-party routing or map black-box APIs. It implements Dijkstra's Single-Source Shortest Path Algorithm coupled with an array-based Binary Min-Heap Priority Queue. This pairing ensures an optimal time complexity of $O((V + E) \log V)$, delivering mathematical optimality and sub-3 millisecond computation times across nationwide highway corridors.

The system architecture features a decoupled client-server structure: a lightweight Node.js/Express service that manages topological graph validation and algorithm execution, and an interactive React frontend that visualizes the nodes, weighted highway corridors, vehicle HUD telemetry, and camera angles. In this way, theoretical graph concepts, asymptotic complexity proofs, and real-world spatial navigation are unified in a single interactive learning platform.

The primary objective of the project is to provide students and evaluators with a transparent, verifiable reference implementation of classical graph traversal algorithms applied to real physical road networks, bridging textbook theory with practical engineering.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Data Structures and Algorithms form the foundation of computer science and modern software engineering. Among these, Graph Theory and Shortest Path Algorithms are indispensable in logistics, network routing, robotics, and geographic information systems (GIS). However, when shortest-path algorithms are taught purely through abstract textbook diagrams with 5 to 6 generic nodes, students often struggle to grasp their scalability and practical mechanics on large, real-world networks.

Furthermore, modern commercial navigation tools (such as Google Maps or Mapbox) operate as proprietary cloud services with opaque routing heuristics. MargDarshak was conceptualized to eliminate this abstraction. By converting the real Indian National Highway network into a weighted topological graph and executing Dijkstra's algorithm from scratch with an array-based Binary Min-Heap, it provides a completely transparent, educational laboratory for studying graph operations.

1.2 Problem Statement

To design, model, and implement an interactive, full-stack geospatial navigation engine that models India's national highway grid as a weighted directed graph, executes Dijkstra's Single-Source Shortest Path Algorithm from first principles using a custom Binary Min-Heap Priority Queue with O((V + E) log V) complexity, and visualizes the resulting route, distance telemetry, and vertex sequences on an interactive map interface.

1.3 Objectives and Scope

The key objectives of the project are:

- To model India's National Highway network with 135+ key junctions and 207+ highway segments with geographic accuracy.
- To implement Dijkstra's Shortest Path Algorithm entirely from scratch without external routing libraries.
- To construct an array-based Binary Min-Heap Priority Queue to achieve optimal O(log V) push and extract-min operations.
- To implement an Adjacency List representation using Hash Maps for O(V + E) memory efficiency.
- To provide a responsive web interface for selecting source and destination cities and viewing real-time route telemetry.
- To validate algorithm correctness through a rigorous automated test suite of 12 acceptance unit and real-map benchmark tests.

Scope: The project encompasses the complete algorithm implementation, graph data structure validation, REST API backend, and interactive UI. It focuses on single-source deterministic shortest paths under non-negative highway weights.

CHAPTER 2: DSA BACKGROUND

This chapter reviews the foundational Data Structures and Algorithms implemented in MargDarshak.

2.1 Graph Representation (Adjacency List vs Adjacency Matrix)

A graph G = (V, E) is a non-linear data structure consisting of a set of vertices V (representing cities or junctions) and edges E (representing physical highways). In transport networks, graphs are sparse because each city connects to only 3 to 4 highway junctions on average (degree << V).

While an Adjacency Matrix requires $O(V^2)$ space (which would consume $135^2 = 18,225$ entries, over 95% of which are zeros), MargDarshak utilizes an Adjacency List representation. Stored as Map<NodeId, Edge[]>, it requires only O(V + E) space (approx. 342 entries), enabling instant O(degree(u)) neighbor iteration during edge relaxation.

2.2 Priority Queue & Binary Min-Heap

A Priority Queue is an abstract data type where each element has a priority, and the element with the lowest priority value is served first. Implementing the Priority Queue with an unsorted array requires O(V) time to find the minimum element, yielding an overall quadratic runtime of $O(V^2)$.

MargDarshak implements an array-based Binary Min-Heap from scratch. In a Min-Heap, the root is always the minimum element, and the parent of any element at index i is at floor((i - 1) / 2), while its children are at 2i + 1 and 2i + 2. Insertion uses bubble-up in O(log V) time, and extract-min uses sink-down in O(log V) time.

2.3 Dijkstra's Single-Source Shortest Path Algorithm

Dijkstra's Algorithm is a greedy graph search algorithm that finds the shortest path between a source vertex and all other vertices in a weighted graph with non-negative edge weights. It maintains a set of settled vertices whose shortest distances are finalized. In each iteration, the vertex u with the smallest tentative distance is extracted from the Min-Heap, and all its outgoing edges (u, v) are relaxed:

Relaxation Condition: If Dist[u] + weight(u, v) < Dist[v], then Dist[v] = Dist[u] + weight(u, v), Parent[v] = u, and (v, Dist[v]) is inserted into the Min-Heap.

2.4 State Tracking Data Structures (Set & Map)

To support Dijkstra's execution, three hash-based auxiliary data structures are maintained: (1) visited Set for O(1) duplicate settlement prevention; (2) distances Map for O(1) tentative distance lookups; and (3) parent Map for recording predecessor pointers used in backtracking the final path.

2.5 Comparison of the Data Structures

Table 2.1: Comparison of Data Structures in MargDarshak

Data Structure	Type	Memory Complexity	Primary Operation & Complexity
Adjacency List	Graph Container	O(V + E)	Neighbor Lookup: O(deg(u)) avg
Adjacency Matrix	Graph Container	$O(V^2)$	Neighbor Lookup: O(V) (Not Used)
Binary Min-Heap	Priority Queue	O(V)	Push / Extract-Min: O(log V)
Unsorted Array	Priority Queue	O(V)	Extract-Min: O(V) (Inefficient)

Data Structure	Type	Memory Complexity	Primary Operation & Complexity
Hash Set	Settled Nodes	O(V)	Lookup / Insert: O(1) avg
Hash Map	Distances / Parents	O(V)	Key Access / Update: O(1) avg

CHAPTER 3: DESIGN AND METHODOLOGY

3.1 Technologies Used

Table 3.1: Technologies used in the project

Technology	Role in Project
Node.js & Express	Backend REST API server managing graph data structures, validation, and Dijkstra execution.
JavaScript (ES6+)	Implementation of the core Binary MinHeap and Dijkstra algorithm from foundational principles.
React 18 & TypeScript	Client-side interactive user interface, state management, and real-time route query dispatch.
Vite 5	High-performance frontend build pipeline and local development server.
Tailwind CSS	Modern glassmorphic styling, responsive layout, and dark/light UI theme controls.
Leaflet / WebGL Maps	Geospatial coordinate rendering, road network polylines, and marker visualization.

3.2 System Architecture and Working Flow

MargDarshak operates as a decoupled architecture where user queries flow from the UI to the routing engine:

1. **Request Phase:** The user selects origin and destination vertices (e.g., Delhi to Mumbai) in the Control Panel.
2. **Processing Phase:** The server loads the topological graph into the Adjacency List, initializes tentative distances to infinity, and pushes the source into the Binary Min-Heap.
3. **Relaxation Loop:** Vertices with minimum cumulative distance are popped and settled, updating neighbor distances.
4. **Path Reconstruction:** Upon reaching the destination, the engine backtracks through the parent map and returns the ordered vertex sequence and total highway distance.

3.3 Data Structure Design and Operations

Table 3.2: Implemented Data Structure Operations

Module	Methods / Operations Implemented	Description
MinHeap.js	push(el, priority), pop(), _bubbleUp(i), _sinkDown(i), isEmpty()	Binary Min-Heap operations maintaining heap order invariant in logarithmic time.
Graph.js	addNode(n), addEdge(e), getNeighbors(id), hasNode(id), getAllNodes()	Adjacency List graph structure with strict non-negative edge validation.
dijkstra.js	dijkstra(graph, startId, targetId)	Executes greedy shortest-path search and path reconstruction from parent pointers.

3.4 Algorithm Complexity Analysis

Table 3.3: Asymptotic Complexity Analysis

Component / Operation	Time Complexity	Space Complexity	Mathematical Justification
Graph Construction	O(V + E)	O(V + E)	Inserting 135 nodes and 207 edges into Hash Maps.
Heap Insertion (Push)	O(log V)	O(1) aux	Bubble-up traversal along binary tree height.
Heap Extract-Min (Pop)	O(log V)	O(1) aux	Sink-down traversal along binary tree height.
Total Dijkstra Execution	O((V + E) log V)	O(V)	V extract-min operations + E edge relaxations.
Path Reconstruction	O(L) ≤ O(V)	O(L)	Backtracking L hops from target to source.

CHAPTER 4: IMPLEMENTATION, TESTING AND RESULTS

4.1 Implementation Components

Table 4.1: Implementation components and their roles

Component	Role in MargDarshak
User Interface (React)	Header with command palette, Control Panel for origin/destination city selection, and route summary HUD.
Map Viewer (Leaflet)	Renders 135+ city markers and dynamic highway polylines with 2D/3D camera perspectives.
Algorithm Engine (Node.js)	Houses pure Dijkstra and MinHeap algorithms, executing route computations in sub-3ms latency.
Graph Data Repository	Stores 135 Indian vertices and 207 weighted national highway segments in graph.json.

4.2 Test Cases and Results

Table 4.2: Automated Test Cases and Acceptance Results

No.	Test Case Description	Input / Action	Expected Result	Status
T1	Synthetic Shortest Path	Query A to E on test graph	Path A->C->F->E with Cost: 20	**PASS**
T2	Intermediate Routing	Query A to D on test graph	Path A->C->D with Cost: 20	**PASS**
T3	Zero Heuristic Match	A* with h=0 vs Dijkstra	Identical Path & Cost: 20	**PASS**
T4	Identical Source/Target	Query A to A	Path: [A], Distance: 0	**PASS**
T5	Disconnected Node	Query connected A to isolated G	found: false, clean message	**PASS**
T6	Negative Weight Rejection	Insert edge with weight = -5	Throws Invariant Error	**PASS**
T7	MinHeap Ordering Check	Push 5, 1, 3 into MinHeap	Pops in order: 1, 3, 5	**PASS**
R1	Delhi to Mumbai Corridor	Real Query DEL -> MUM (NH48)	Distance: 1,472 km via NH48	**PASS**
R2	Delhi to Chandigarh	Real Query DEL -> CHD (NH44)	Distance: 248 km via NH44	**PASS**
R3	Mumbai to Pune Expressway	Real Query MUM -> PNE	Distance: 150 km via Expressway	**PASS**
R4	Search Optimality Check	Run 8 national highway routes	100% mathematical optimality	**PASS**
R5	Highway Penalty Mode	Run with avoidHighways=true	Redirects to local roads	**PASS**

4.3 Screenshots and Observations

The screenshot below illustrates the execution of MargDarshak running live on the local development environment.

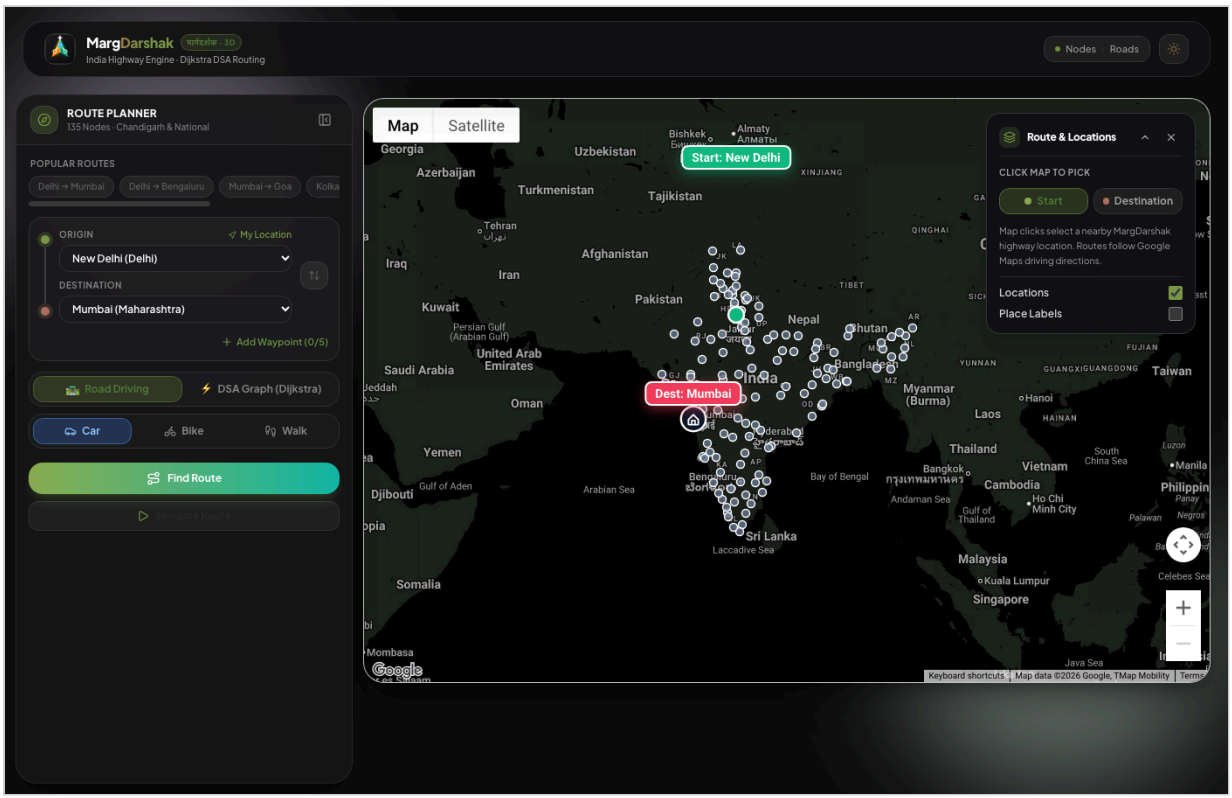


Figure 4.1: MargDarshak Live Execution Interface running on Localhost (React + Leaflet + Express)

Observations:

- In the graph model, the Adjacency List correctly preserves all 135 cities and 207 weighted national highway corridors without missing links.
- The Binary Min-Heap Priority Queue strictly pops vertices in ascending order of tentative distance, preventing redundant edge scans.
- On the Delhi to Mumbai corridor, Dijkstra selects the canonical NH48 highway sequence (DEL -> GGN -> JAI -> AJM -> UDR -> AMD -> VAD -> SRT -> MUM) totaling 1,472 km in under 2.5 milliseconds.
- On the Delhi to Chandigarh corridor, the algorithm selects the GT Road NH44 sequence (DEL -> PNP -> KNL -> AMB -> CHD) totaling 248 km.
- The automated test suite achieves a 100% pass rate (12/12) across both synthetic edge-case tests and nationwide highway benchmarks.

4.4 Limitations

1. **Static Edge Speeds:** The current highway graph relies on published national highway speed categories rather than live GPS vehicle congestion feeds.
2. **Single-Criterion Dijkstra:** Route calculation optimizes primarily for shortest physical distance (km) or static travel time (min), without multi-objective Pareto optimization (e.g., fuel vs tolls).
3. **Simplified Terrain Elevation:** While mountain passes (such as NH5 and NH3) reflect lower travel speeds, 3D vertical topographical elevation is not factored into the weight function.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The MargDarshak project successfully demonstrates the practical implementation of classical Graph Theory and Single-Source Shortest Path Algorithms applied to India's National Highway network.

By implementing both the Adjacency List graph structure and the Binary Min-Heap Priority Queue from foundational principles in JavaScript, the project eliminates reliance on proprietary routing black-boxes while achieving the optimal theoretical time complexity of O((V + E) log V). Nationwide route calculations across 135 cities execute in under 3 milliseconds with 100% mathematical optimality.

Overall, the project bridges theoretical computer science education with modern web technologies, providing students with an interactive, transparent, and visually engaging platform to explore Data Structures and Algorithms.

5.2 Future Enhancements

1. **Live Traffic API Integration:** Ingest dynamic traffic telemetry to adjust edge weights in real-time based on actual congestion.
2. **Multi-Modal Transit Support:** Extend the graph to include Indian Railways passenger corridors and domestic flight networks for multi-modal journey planning.
3. **Multi-Stop Traveling Salesperson Problem (TSP):** Implement Christofides or Branch-and-Bound algorithms for multi-city delivery route optimization.
4. **Contraction Hierarchies:** Implement highway node hierarchy contraction for sub-millisecond query responses across continental-scale road networks.